MONITOR Z · L15.1

Clase 1 — Del fuente al programa que corre

Cuaderno de estudio · versión del 2026-10-26

Margen derecho y renglones libres para tus notas. Las láminas son las mismas de la carpeta **_visual**. Si una lección se corrige, se regenera este cuaderno con la fecha nueva.

L15 · Clase 1 — Del fuente al programa que corre

La CPU sólo entiende bytes. Entre el texto que escribís y el monitor corriendo hay **tres etapas**, y cada una la hace un programa distinto de z/OS.

> 📚 Fuentes: tus JCL `ZTKMONVA` y `ZTKMONVR` (`monitor_z/jcl/`). El listado del ensamblado y el mapa del binder de **JOB34775** (22-10), en `monitor_z/salidas/JOB34775/`. *HLASM Programmer's Guide* (SC26-4941).

1. Las tres etapas

Etapa	Programa de z/OS	Entra	Sale	Paso de tu JCL
1. Ensamblar	ensamblador HLASM (`ASMA90`)	el fuente (texto)	módulo **objeto** + listado	`ASM` de `ZTKMONVA`
2. Enlazar	**binder** (`IEWL`)	objetos + piezas de biblioteca	**load module**	`LKED` de `ZTKMONVA`
3. Cargar	cargador de z/OS	el load module	el programa **en memoria, corriendo**	`EXEC PGM=ZTKMON` de `ZTKMONVR`

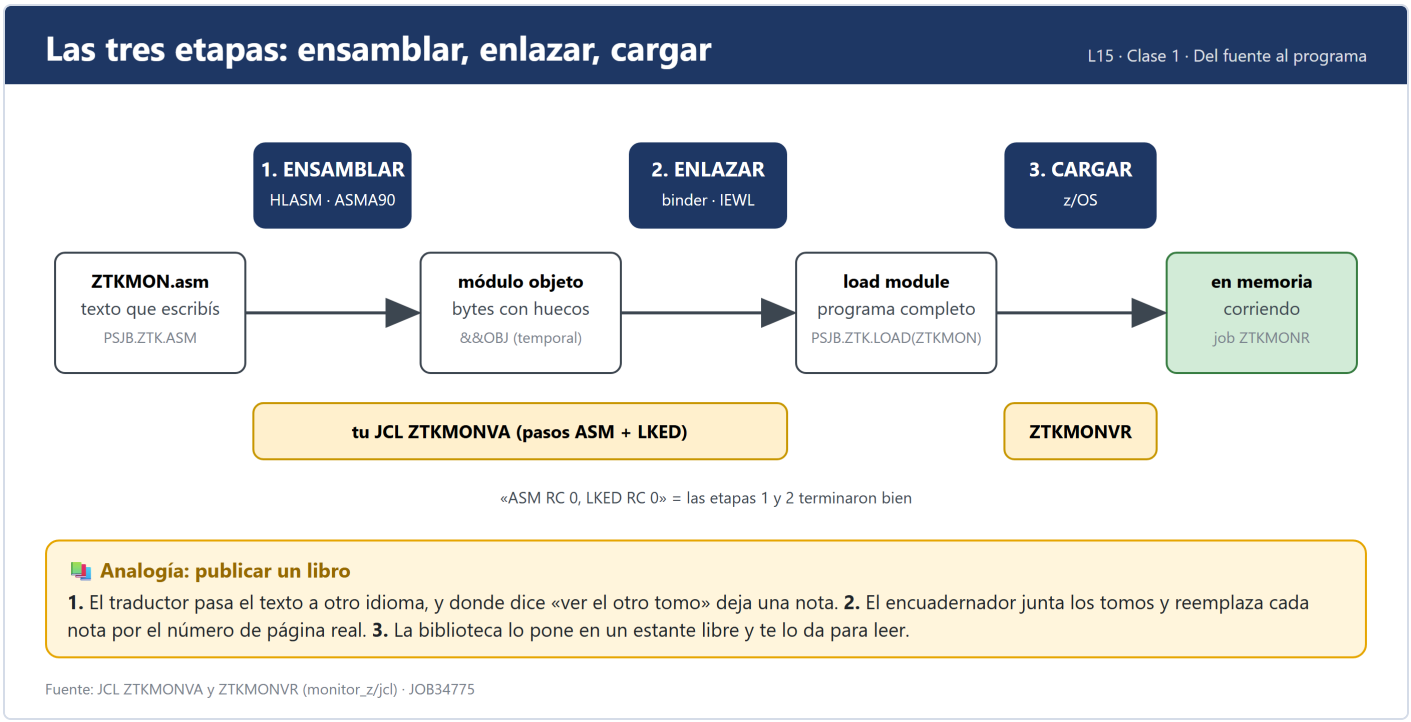


Lámina L15-01_tres_etapas

2. Ensamblar: de texto a bytes

- El ensamblador lee `PSJB.ZTK.ASM(ZTKMON)` y traduce **cada instrucción a su código de máquina**.
- Ejemplos reales de JOB34775:
  - `LR R12,R15` → `18CF` (2 bytes);
  - `L R0,NREP` → `5800 BE2C` (4 bytes).
- Las macros de IBM (`CALL`, `STORAGE`, `IEFSSREQ`…) salen de `SYS1.MACLIB` y `SYS1.MODGEN`. Por eso están en el `SYSLIB` del paso `ASM`.
- **Lo que no está en tu fuente queda como un hueco anotado.** `CALL EZASOKET` se ensambló como `DC V(EZASOKET)` con los bytes `00000000` (posición `000DD8`). Además, el ensamblador anotó `EZASOKET` en su lista de símbolos externos como **ER** (*external reference*): «acá falta una dirección de afuera».
- **Salen dos cosas:**
  - el **objeto** (`SYSLIN`, el temporal `&&OBJ`), que es lo que sigue a la etapa 2;
  - el **listado** (`SYSPRINT`), que es tu herramienta para ver en qué se convirtió cada línea y dónde quedó.

Etapa 1 · Ensamblar: de texto a bytes

L15 · Clase 1 · Del fuente al programa

Lo que escribiste (fuente)	Lo que salió (bytes)	Qué significa
`LR R12,R15`	`18CF`	`18` = LR · `C` = R12 · `F` = R15
`L R0,NREP`	`5800 BE2C`	`58` = L · `0` = R0 · base `B` = R11 · desplazamiento `E2C`
`CALL EZASOKET,(…)`	`00000000`	se expande a `DC V(EZASOKET)` : **la dirección quedó en cero**. Es el hueco anotado



ER = *external reference*: el ensamblador anota «acá falta la dirección de algo que no está en este fuente». No es un error: es un pedido para la etapa 2.

Fuente: Listado ASM de JOB34775: sentencias 10127 (LR), 10244 (L R0,NREP), 11311 (DC V(EZASOKET) en 000DD8) y el ESD

Lámina L15-02_ensamblar

3. Enlazar: el binder arma el programa

- El binder recibe tu objeto (`SYSLIN`) y busca lo que falta en las bibliotecas de `SYSLIB`: para vos, `TCPIP.SEZATCP`.
- Pone las piezas una detrás de otra y **llena cada hueco** con la dirección real. El mapa de JOB34775:

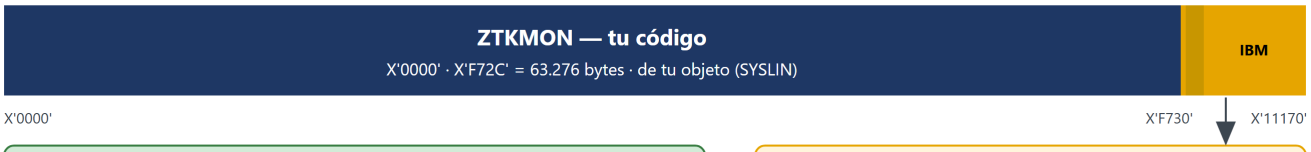
Desde	Pieza	Largo	De dónde vino
X'0000'	`ZTKMON`	X'F72C' = 63.276 bytes	tu objeto
X'F730'	`EZBOECPR`	X'100'	`TCPIP.SEZATCP(EZASOKET)`
X'F830'	`EZASOKET`	X'3D5'	`TCPIP.SEZATCP(EZASOKET)`
X'FC08'	`EZASOKFN`	X'1568'	`TCPIP.SEZATCP(EZASOKET)`
total		**X'11170' = 70.000 bytes**	

- **Lo guarda** en `SYSLMOD` = `PSJB.ZTK.LOAD(ZTKMON)`, como *LOAD MODULE*, AMODE 31 y RMODE 24, con punto de entrada en `ZTKMON`, offset 0.
- `COND=(4,LT,ASM)`: si el `ASM` terminó con un RC **mayor que 4**, el `LKED` no corre. No se arma un programa con errores.

Etapa 2 · Enlazar: el binder arma el programa

L15 · Clase 1 · Del fuente al programa

El load module ZTKMON, pieza por pieza (mapa del binder, JOB34775)



Y llena los huecos:
el `DC V(EZASOKET)` que valía 00000000 pasa a valer la dirección real de EZASOKET dentro del programa

De TCPIP.SEZATCP(EZASOKET):
EZBOECPR en X'F730' · X'100' bytes
EZASOKET en X'F830' · X'3D5' · EZASOKFN en X'FC08' · X'1568'

Línea del JCL (paso LKED)	Qué le dice al binder
`EXEC PGM=IEWL,COND=(4,LT,ASM)`	Arrancá... salvo que el ASM haya dado RC **mayor que 4**: no se arma un programa con errores
`SYSLIN DD &&OBJ`	Tu objeto, el que dejó la etapa 1
`SYSLIB DD TCPIP.SEZATCP`	Dónde buscar lo que falta (EZASOKET)
`SYSLMOD DD PSJB.ZTK.LOAD(ZTKMON)`	Dónde guardar el programa terminado: **LOAD MODULE, AMODE 31, RMODE 24**, entrada en ZTKMON (offset 0)

Fuente: Mapa y SAVE OPERATION SUMMARY del paso LKED de JOB34775 · JCL ZTKMONVA

Lámina L15-03_enlazar

4. Cargar: z/OS lo pone en memoria y lo arranca

- `EXEC PGM=ZTKMON` + `STEPLIB DD DSN=PSJB.ZTK.LOAD`: z/OS busca el programa **primero** en tu biblioteca.
- Lo copia a un **lugar libre de la memoria**. Como es RMODE 24, va debajo de los 16 MB. **El programa no elige la dirección.**
- Le pasa el control y deja en **R15** la dirección donde empieza. Por eso la primera instrucción útil de ZTKMON es `LR R12,R15`: guarda esa dirección para usarla de base.
- El `PARM='ZTKD* 5 30 8010'` le llega al programa por **R1**.

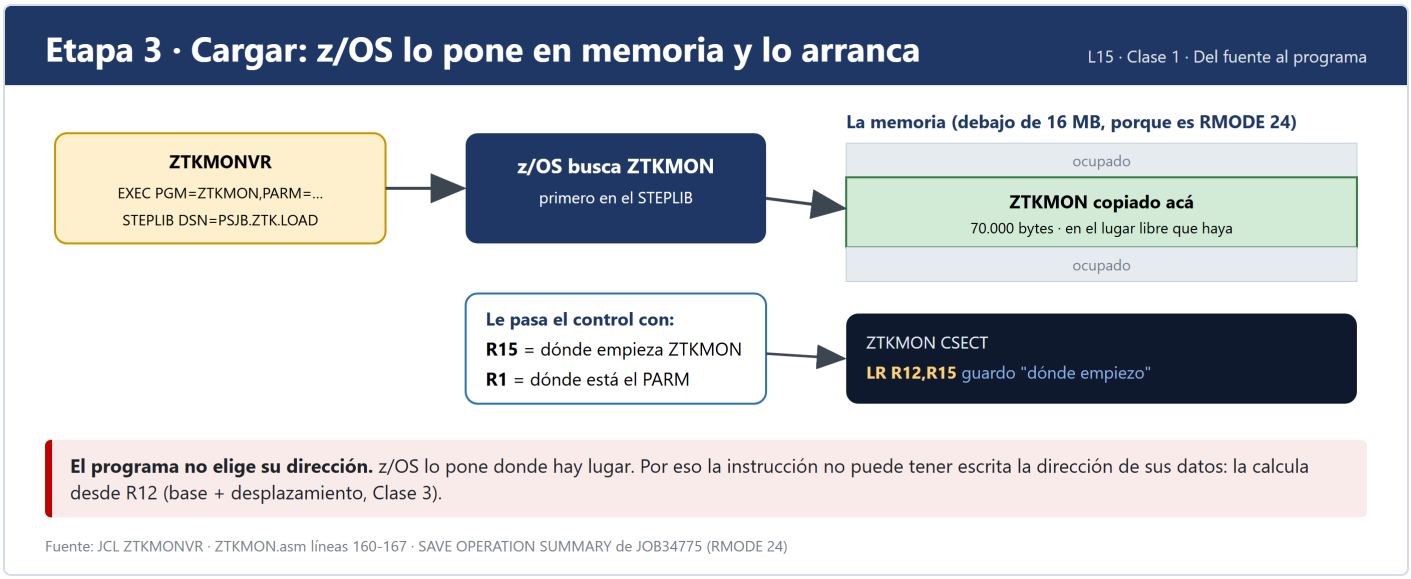


Lámina L15-04_cargar

5. Tus JCL hacen exactamente esto

Tu JCL	Línea	Etapa	Qué indica
`ZTKMONVA`	`//ASM EXEC PGM=ASMA90`	1	el ensamblador
	`//SYSIN DD DSN=PSJB.ZTK.ASM(ZTKMON)`	1	el fuente
	`//SYSLIB DD DSN=SYS1.MACLIB (+ MODGEN )`	1	las macros de IBM
	`//SYSLIN DD DSN=&&OBJ`	1 → 2	el objeto, que pasa al paso siguiente
	`//LKED EXEC PGM=IEWL`	2	el binder
	`//SYSLIB DD DSN=TCPIP.SEZATCP`	2	de dónde sacar `EZASOKET`
	`//SYSLMOD DD DSN=PSJB.ZTK.LOAD(ZTKMON)`	2	dónde guardar el programa
`ZTKMONVR`	`//MONITOR EXEC PGM=ZTKMON,PARM=...`	3	qué cargar y con qué parámetros
	`//STEPLIB DD DSN=PSJB.ZTK.LOAD`	3	dónde buscarlo

6. Qué cambia con los módulos

- **Hoy:** 1 pieza tuya (`ZTKMON`) + 3 de IBM.
- **Con módulos:** varias piezas tuyas (`ZTKMON`, `ZTKJES`, `ZTKRED`…) + las mismas 3 de IBM.
- **Etapa 1:** cada pieza tuya se ensambla **por separado**, y cada una deja sus huecos (`V(ZTKJES)`…).
- **Etapa 2:** el binder las une en **un solo** `PSJB.ZTK.LOAD(ZTKMON)` y llena los huecos **entre tus piezas**, como hoy lo hace con `EZASOKET`.
- **Etapa 3:** no cambia. Se sigue sometiendo `ZTKMONVR`; el operador y el puente no notan nada.

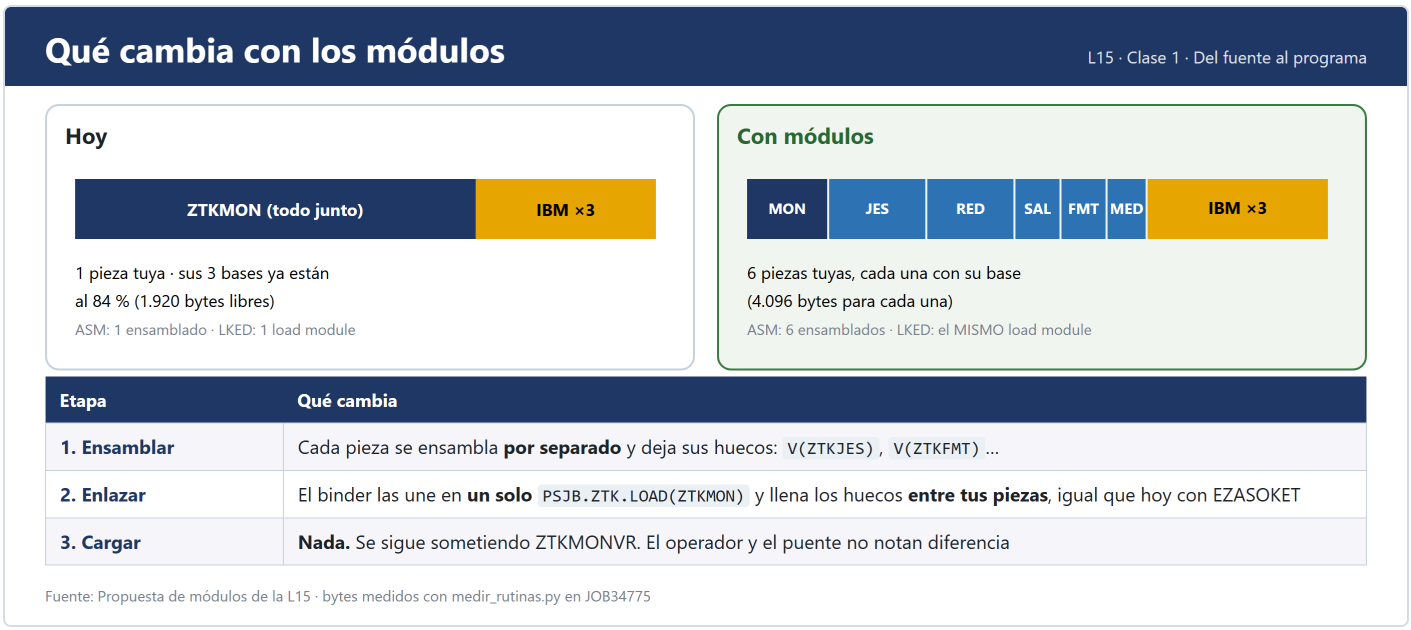


Lámina L15-05_modulos_hoy_y_manana

7. Resumen

	Ensamblar	Enlazar	Cargar
Quién	HLASM	binder	z/OS
Analogía	el traductor que deja «ver el otro tomo»	el que atornilla el mueble	el que lo pone en la sala y lo enciende
Lo que resuelve	texto → bytes	huecos → direcciones	dónde vive → R15
Lo que **no** sabe	dónde quedan las piezas de afuera	dónde se va a cargar	—

8. Preguntas de comprobación

1. ¿Qué tres etapas hay entre `ZTKMON.asm` y el monitor corriendo, y qué programa hace cada una?
2. En tu listado, `DC V(EZASOKET)` dio `00000000`. ¿Por qué cero, y quién lo completa?
3. En el mapa del binder, ¿qué pieza empieza en X'F830' y de qué biblioteca vino?
4. ¿Qué pasa con el paso `LKED` si el `ASM` termina con RC 8? ¿Qué línea del JCL lo decide?
5. ¿Por qué ZTKMON hace `LR R12,R15` apenas arranca?
6. Si separamos las rutinas de fechas en una pieza `ZTKFMT`, ¿en qué etapa se unen con el resto, y quién llena el hueco de la llamada?

Mis respuestas

Notas